Modele de Calcul

2024

Cuprins

1 Motivație **1**

2 Obiective **1**

3 Probleme și Algoritmi **1**

4 Mașina Turing **2**

4.1 Mașina Turing Nedeterministă 4

5 Teza Church-Turing **5**

1 Motivație

În următoarele capitole ne propunem să abordăm două probleme fundamentale în știința calculatoarelor: identificarea problemelor care NU pot fi rezolvate, indiferent de câte resurse (timp, spațiu) suntem dispuși să investim, respectiv recunoașterea acelor probleme pentru care nu cunaștem soluții care să fie în același timp **eficiente**, respectiv **corecte** pentru **orice** intrare.

Înainte de a aborda cele două subiecte, vom introduce un model de calcul de referință (Mașina Turing), a cărui execuție să fie ușor de analizat dar suficient de expresiv pentru a putea rezolva orice problema care poate fi rezolvată. În acest material ne propunem să sintetizăm informațiile discutate la curs pe această temă.

2 Obiective

Ne vom familiariza cu următoarele concepte:

- Ce este o problemă? Ce este un algoritm?
- Mașina Turing deterministă. Variante ale unei MT.
- Mașina Turing nedeterministă.
- Teza Church-Turing.

3 Probleme și Algoritmi

Probleme Formalizăm conceptul de problemă ca o funcție care asociază unui element dintr-o mulțime de intrări $\mathbb{I}$ un element dintr-o mulțime de rezultate $\mathbb{O}$ ($f : \mathbb{I} \to \mathbb{O}$).

În practică întâlnim multe tipuri de probleme: de decizie, optimizare, generare, numărare, etc. De exemplu, fie două locații A și B dintr-un graf neorientat. Am putea dori să răspundem la următoarele întrebări:

- PATH: Există drum de la A la B? (decizie)
- SHORTEST-PATH: Care este drumul de cost minim? (optimizare)
- K-SHORTEST-PATH: Care este al K-lea drum de cost minim? (optimizare)

- GEN-PATH: Care sunt toate drumurile dintre A și B? (generare)
- COUNT-PATH: Câte drumuri sunt între A și B? (numărare)

În continuare, pentru a simplifica discuția, vom face două alegeri:

1. Vom reprezenta elementele din mulțimea de intrări ca un secvență finită de caractere. Prin convenție, **folosim Σ pentru a descrie alfabetul folosit**, respectiv Σ^* pentru a reprezenta o secvență de caractere. În particular, din rațiuni practice, vom alege să reprezentăm numerele primite la intrare folosind o **codificare binară** ($9 \implies '1001'$).
2. Ne vom concentra pe rezolvarea **problemelor de decizie**, pentru care mulțimea rezultatelor este $\{True, False\}$[1].

Mulțimea de adevăr a unei probleme de decizie este compusă din acele intrări pentru care funcția asociată ar întoarce valoarea $True$. Deoarece reprezentăm intrările problemei ca o secvență de caractere, vom folosi și termenul de **limbaj** pentru a descrie mulțimea de adevăr.

Algoritmi O rezolvare a unei probleme este un algoritm compus dintr-o o secvență **finită** de instrucțiuni bine definite (care pot fi urmate mecanic) pentru a obține un rezultat **unic** (mereu același pentru aceleași date de intrare), într-un număr **finit de pași** și consumând o cantitate **finită de resurse** (ex: memorie, etc.). Dacă o funcție are asociată o rezolvare spunem că este **calculabilă efectiv**.

4 Mașina Turing

Automat finit Una dintre cele mai simple modalități de a specifica un algoritm este prin intermediul unui automat finit. Astfel, un automat finit este definit prin:

- **O mulțime finită de stări** (Q), între care se află *o stare inițială* q_0, respectiv o mulțime de stări finale (ex: cel puțin o stare care să semnifice că automatul *a acceptat intrarea*, respectiv o stare care să semnifice că automatul *a respins intrarea*).
- **O funcție de tranziție** ($\delta : Q \times \Sigma \to Q$) care specifică cum se modifică starea automatului pentru un anumit caracter primit la intrare.

Procesul de calcul se termină dupa procesarea tuturor caracterelor primite la intrare.

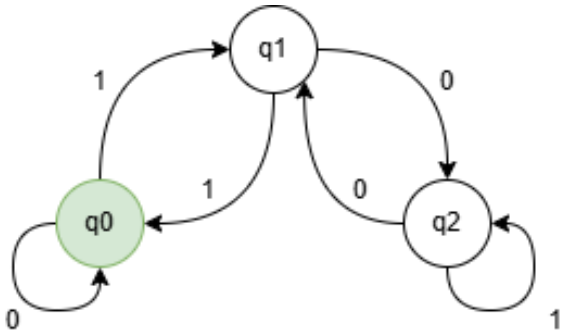


Figura 1: Un automat finit care verifică dacă un număr primit la intrare (reprezentat ca o secvență binară) este divizibil cu 3. Automatul are trei stări (q_0, q_1, q_2) care sunt folosite pentru a determina restul impărțirii la 3 a secvenței parcurse. Dacă automatul termină procesarea în starea q_0, atunci considerăm că a *acceptat* intrarea, altfel considerăm că aceasta a fost *respinsă*[2].

[1] În multe situații practice, putem reformula o problemă de optimizare (ex: '*Care este costul minim pentru a realiza X?*') ca o problemă de decizie ('*Putem realiza X cu un cost de cel mult Y?*'). Astfel, dacă putem rezolva eficient problema de decizie, respectiv să explorăm eficient spațiul valorilor posibile Y, am avea o soluție viabilă și pentru problema inițială de optimizare. De exemplu, putem spune că problema calculării drumului cu cele mai puține muchii intermediare între două noduri dintr-un graf orientat este *cel puțin la fel de grea* ca problema verificării dacă există un drum între A și B cu cel mult K muchii. Prin urmare, dacă știm că problema de decizie este *grea*, în sensul că nu am descoperit o soluție eficientă pentru a o rezolva, și problema corespunzătoare de optimizare va fi *grea*.

[2] Explicație: Observăm că citirea unui '0' din secvența de intrare este echivalentă cu dublarea numărului curent, respectiv citirea unui '1' este echivalentă cu dublarea numărului curent și adunarea unui 1.

Automat push-down Există limbaje care nu pot fi recunoscute de un automat finit. De exemplu, nu putem folosi un automat finit pentru a recunoaște o secvență arbitrar de mare de paranteze închise corect (ex: '(()))()'). Am putea rezolva această problemă dacă am avea acces la o structură de tip stivă pe care putem scrie sau șterge un număr arbitrar de simboluri. Acest tip de automat este cunoscut în literatură ca un automat push-down (APD). În continuare însă există limbaje care nu pot fi recunoscute nici de un APD, precum $\{0^n1^n2^n \mid n \in \mathbb{N}\}$.

Mașina Turing Deterministă O mașina Turing (MT) extinde noțiunea de automat finit printr-o bandă de memorie (T - tape), formată din celule. Banda se poate extinde arbitrar de mult la dreapta, adăugând celule pe măsură ce acestea sunt necesare. La începutul execuției banda conține pe primele poziții datele de intrare. Celulele suplimentare sunt marcate inițial cu un simbol generic (ex: BL - blank).

Astfel, vom modifica specificația funcției de tranziție pentru a permite mașinii să modifice valoarea de la poziția indicată de un cap de citire, respectiv să mute capul o poziție la stânga sau la dreapta:

$$\delta : Q \times \Sigma \to Q \times \Sigma \times \{Left, Right\}$$

De exemplu, dacă $\delta(q_1, 0) = (q_2, 1, Left)$ atunci, dacă MT se află în starea q_1, iar la poziția curentă a capului de control se află scrisă valoarea '0', MT va trece în starea q_2, va scrie valoarea '1' la poziția curentă și va muta capul de bandă un pas la stânga.

Prin urmare, pentru a descrie complet **configurația** unei MT de la un pas vom avea nevoie de:

- Starea în care se află MT (Q).
- Poziția capului de bandă (B).
- Valorile scrise pe banda (T).

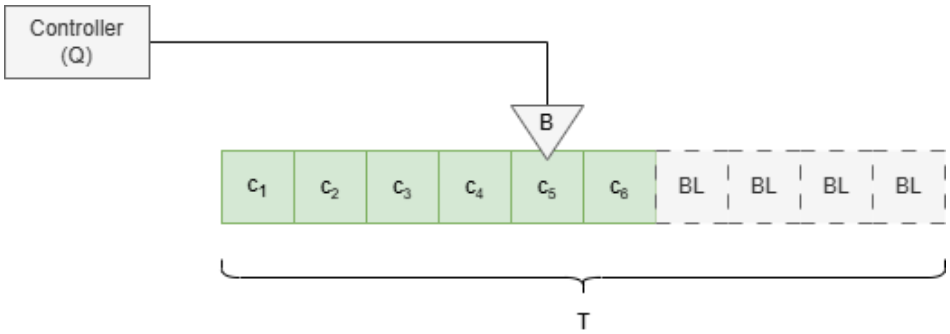


Figura 2: O reprezentare grafică a unei MTD cu o singură bandă. În funcție de starea curentă, unitatea de control a MT modifică valoarea aflată la poziția capului de bandă, respectiv mută capul de bandă un pas la stânga sau la dreapta.

Dacă MT ajunge într-o stare finală (de acceptare sau respingere), procesul de calcul se termină.

Observație 1. *Am ales să studiem Mașina Turing și nu un alt model de calcul la fel de expresiv pentru că este ușor să analizăm riguros execuția sa. O descriere completă a unei MT presupune detalierea fiecărei stări, respectiv a funcției de tranziție. Acest exercițiu însă este destul de elaborat, chiar și pentru probleme relativ simple. Alternativ, putem încerca o descriere de nivel înalt a implementării unei MT, ca în exemplul de mai jos.*

Exemplu 1: Este secvența primită la intrare palindrom?

Descriere rezolvare (limbaj natural):

1. Procesarea începe cu primul caracter scris pe bandă.
2. Comparăm primul caracter cu ultimul caracter:
   - Dacă cele două caractere sunt la fel, le înlocuim cu simbolul default (blank). Continuăm cu următorul caracter care este diferit de simbolul blank.
   - Dacă nu sunt la fel, respingem intrarea.
3. Dacă mai există un singur caracter pe bandă (sau nu mai există niciunul), trecem în starea de acceptare.

Mașina Turing Universală Cum ar fi dacă ar trebui să proiectăm un calculator nou pentru fiecare problemă nouă pe care vrem să o rezolvăm? Alan Turing a intuit că acest lucru nu este necesar, demonstrând ca putem defini o Mașină Turing care să primească la intrare codificarea unei alte MT, respectiv o intrare validă pentru aceasta, și să simuleze execuția MT pe intrarea respectivă.

Variante ale unei MT Putem modifica definiția unei mașini Turing fără să afectăm **expresivitatea**[3] acesteia. Astfel, putem întâlni definiții alternative precum:

- În plus față de acțiunea de a muta capul de bandă la stânga sau dreapta permitem capului de bandă să rămână staționar.
- Banda se poate extinde arbitrar de mult în ambele direcții.
- Pe lângă banda obișnuită putem avea o bandă suplimentară 'read-only' (pentru care nu putem modifica valoarea celulelor), pe care se află datele de intrare.
- Permitem existența unui număr finit de benzi care pot fi manipulate simultan.

4.1 Mașina Turing Nedeterministă

Observăm că, dintr-o configurație validă a MT **o singură** tranziție este posibilă. Cu alte cuvinte, procesul de calcul este **determinist** - există o unică secvență posibilă de configurații. Astfel, dacă într-un punct din procesul de rezolvare avem K alegeri posibile (unde K este un număr cunoscut, finit), va trebui să le investigăm pe rând. Dar dacă MT ar putea *ghici* care este opțiunea care conduce spre starea de acceptare a intrării?[4] Oare am putea rezolva *eficient* mai multe probleme?[5]

Definim un model de calcul teoretic care ne oferă această posibilitate. Astfel, Mașina Turing nedeterministă (MTN), extinde definiția funcției de tranziție:

$$\delta : Q \times \Sigma \to \mathbb{P}(Q \times \Sigma \times \{Left, Right\})$$

De exemplu, pentru perechea $(q_1, 0)$ am putea defini două tranziții posibile: $(q_2, 1, L)$ sau $(q_3, 0, R)$.

Lema 1. *Pentru fiecare Mașină Turing nedeterministă există o Mașină Turing deterministă echivalentă.*

[3] Prin expresivitate înțelegem limbajele care pot fi acceptate, respectiv problemele pe care le poate rezolva.

[4] Alternativ: Dacă MT ar putea analiza, fără un cost adițional, consecințele tuturor alegerilor pentru a determina care este alegerea optimă?

[5] Această întrebare reprezintă punctul de plecare pentru dilema, dezvoltată în cursurile următoare, dacă putem *rezolva* eficient toate problemele a căror soluție poate fi *verificată* ușor.

5 Teza Church-Turing

Teza 1: Church-Turing

Orice funcție este **efectiv calculabilă** dacă și numai dacă poate fi calculată de o **mașină Turing**.

Prin teza Church-Turing stabilim, fără să demonstrăm, o legătură între un concept filozofic general (calculabilitatea efectivă) și un proces definit riguros (calculabilitatea în sens Turing). Pe lângă MT, propusă de Alan Turing, cercetătorii au propus de-a lungul timpului mai multe modele de calcul. De exemplu, Calculul Lambda, introdus de Alonzo Church în aceeași perioadă, stă la bază limbajelor de programare funcționale moderne. Nu s-au descoperit însă modele de calcul mai expresive, care să permită calcularea mecanică a unei funcții care să nu poate fi calculată de o MT.

Spunem că un model de calcul este **Turing-complete** dacă poate fi folosit pentru a simula execuția unei MT arbitrare.

Limbajele de programare folosite uzual sunt echivalente din punct de vedere al calculabilității: putem converti un program arbitrar scris în Python într-un program Java sau invers. Vom constata însă că această expresivitate vine și cu un compromis major în abilitatea noastră de a verifica corectitudinea unui program scris într-un limbaj de programare Turing-complete.

References

[1] Thomas H Cormen et al. *Introduction to algorithms*. MIT press, 2022.

[2] Neil D Jones. "Computability and complexity from a programming perspective". In: *Proof and System-Reliability* (2002), pp. 79–135.

[3] Michael Sipser. *Introduction to the Theory of Computation*. English. 3rd ed. Cengage Learning, 2012. ISBN: 978-1133187790.